TREATING QASM AS WASM

A Reference Architecture and Validation Study for Hybrid Graph Computation

Classical graph aggregation, spiking control, and quantum-circuit simulation
with a reproducible test suite and a scoped mutation-testing method

Revision 4

Written by Aaron Michael Weis
ORCID: https://orcid.org/0009-0002-1906-148X
OneiroNexus

8 November 2026 | Original manuscript: September 2026

Abstract

Evidence and contribution. The final source package and historical ledgers are available. Revision 4 distinguishes those records from its fresh suite run and independent mathematical checks. Earlier baseline archives and some original probe scripts remain unavailable; their results are retained as historical reports. The source code is released under MIT and this paper under CC BY 4.0.

This study examines a software interface analogy: load a circuit description once, then invoke it through an explicit input/output contract, while keeping classical state updates in a separate WebAssembly module. The supplied scaffold combines sparse graph aggregation, a leaky-integrate-and-fire (LIF) activity gate, and a two-qubit circuit evaluated by a classical statevector simulator. It is an untrained, toy-scale reference system. The title does not describe a compiler from QASM to WASM or a demonstrated quantum speedup.

The implementation includes a restricted OpenQASM 2.0 parser, a NumPy reference implementation, and a C LIF kernel compiled to wasm32. A fresh Revision-4 run of the supplied package completed all ten test modules with 1,286 passing check entries and zero failures, using the included WASM binary. The source archive also contains the original suite and mutation ledgers. Mathematical derivations, implementation tests, and historical campaign results are distinguished throughout; passing a finite test suite does not establish correctness for every input.

The earlier manuscript reports an AI-assisted review that injected eighteen selected defects. Table 3 records seven kills and eleven survivors: eight pipeline-composition mutants, one LIF threshold-boundary mutant, one RZ-gate mutant and one backend-invocation mutant. It also reports fixes for malformed-circuit handling and a C-core buffer overrun. The final regression tests have now been rerun successfully; the earlier before-fix archive and the original eighteen-mutant harness are not included, so that historical comparison has not been reproduced separately.

The operator-based method applies six mutation families and records each mutant, selected tests, outcome and survivor verdict. The supplied final ledger contains 446 mutants: 437 kills and nine survivors. Twelve C mutants are already included in those operator totals, resolving the double-counting ambiguity in Revision 3 Table 5. The resulting 98.0% is a detection score for that declared campaign, not an accuracy, reliability or proof percentage. Kills include test failures and crashes; survivor verdicts require substantive review.

The transferable contribution is a practical way to connect architectural claims to tests, then examine whether those tests detect selected incorrect implementations. The quantum primitive has a direct classical cosine baseline; the whole tick retains node-count-dependent work; the fixed three-dimensional projection has no established low-distortion or sharding guarantee. Hardware execution, trained models, production performance and the proposed large-scale roadmap remain future work.

Table of Contents

1. Introduction

2. Background and Related Work

3. System Architecture

4. Methodology: Adversarial Validation

5. Implementation

6. Empirical Results

7. Historical AI-Assisted Review

8. The Method, Systematized

9. Discussion

10. Toward Scale: A Staged Roadmap

11. Limitations

12. Conclusion

References

1. Introduction

1.1 Motivation and Origins

The project grew from an interest in combining graph structure, event timing and quantum-circuit descriptions in a system that another researcher could inspect and extend. The immediate engineering question is narrow: can these parts be given explicit interfaces, composed into a small running pipeline, and checked with tests that examine both local numerical behavior and the connections between components?

The scope is a supplied reference implementation and its experiment records. Revision 4 checks those records against the manuscript and repeats the current test suite. The distinction between an implemented toy system and the much larger proposed deployment remains central: no result here establishes operation at 140 million to 20 billion nodes.

Revisions 1 through 3 describe construction, an AI-assisted review and the addition of automated mutation testing. Revision 4 corrects terminology, reconciles the mutation table with the supplied ledger, makes the actual dataflow explicit, and adds a fresh reproducibility record and further derivations. Historical claims about earlier program versions remain attributed to those records when their exact baseline is not available. An additional model review is not independent human peer review.

1.2 The Central Analogy: QASM as WASM

WebAssembly is a portable code format with a sandboxed execution model [12]. Reproducibility depends on the module's initial state, inputs, imports and runtime configuration; it is not an unconditional guarantee that repeated calls with the same arguments return the same value. OpenQASM describes quantum circuits and their classical interactions [10]. Actual measurements follow the relevant probability distribution, whereas a classical simulator can compute deterministic statevector expectations and generate reproducible pseudorandom shot samples. The distinction concerns execution semantics, state and measurement, rather than a universal deterministic-versus-nondeterministic split between the two languages.

The analogy implemented here is a division of responsibility. A Wasm-compiled LIF core performs the classical state update, while a separately invoked OpenQASM-subset simulator evaluates a small circuit when a spike triggers it. Quantinuum's decoder workflow is cited as related precedent [11], but this scaffold does not implement that system's hardware interface or timing behavior. The QasmResult API names exact statevector-derived expectations and sampled estimates separately; this makes the distinction visible but does not by itself prevent callers from confusing the fields.

1.3 Thesis Statement

This thesis advances four claims. First, the supplied scaffold combines a classical LIF control core compiled to Wasm with graph aggregation and a quantum-circuit-inspired reference map evaluated by a classical simulator. It demonstrates an architectural separation, with no quantum hardware execution or quantum

computational advantage established. Second, comparison with separately derived mathematics and reference implementations can expose defects that internal consistency checks miss. The final suite has been rerun at its documented scope; earlier experimental versions remain historical reports.

Third, a test suite's claims about the defects it detects can themselves be examined by injecting selected defects. In the reported eighteen-mutant campaign, seven mutants were killed and eleven survived. Eight survivors concerned pipeline composition, while the other three concerned a threshold boundary, rz behavior and actual backend invocation. The manuscript reports that subsequent tests killed all eighteen.

Fourth, an operator-based campaign makes defect injection repeatable within an explicit scope and can expose gaps missed by a hand-selected campaign. Its score depends on the selected operators, sites, inputs, exclusions and oracles. The supplied final ledger supports 437 kills among 446 mutants. The reported earlier baseline of 410 among 464 describes a different program and test state, so the two percentages are not a comparison over one fixed set of mutants.

1.4 Contributions

- An OpenQASM 2.0-subset simulator exposed through a load/call interface. It distinguishes ideal statevector expectations from sampled estimates, using NumPy and Python rather than a quantum SDK. Measurement syntax is validated but is not executed with general OpenQASM measurement semantics; sampling reads all qubits at the end.
- A classical LIF step implemented in Python and in C compiled to wasm32, tested for numerical equality and matching spike flags on specified isolated, feedback-coupled and capacity scenarios. The included WASM binary was executed again for Revision 4; a clean compiler rebuild is a separate reproducibility requirement.
- A sparse gather/scatter GNN aggregation layer, reportedly checked against a naive reference on fixed and randomized graphs, and a GraphSAGE-style nodewise layer tested for permutation equivariance: relabeling nodes should relabel the output in the same way. These finite tests support the tested cases; an algebraic identity is required for a general proof.
- A fusion layer producing three projected embedding coordinates and one simulation-age field. For a Gaussian projection with entries of variance 1/3, squared norms are unbiased over projection randomness. The reported Monte Carlo checks test that expectation identity; they do not establish low distortion or useful sharding geometry for the single fixed three-dimensional projection.
- An end-to-end orchestrator whose composition is validated twice over: every quantum-triggered update produced by a live multi-tick run is re-derived from closed-form mathematics, and — since the revision — every other quantity the tick produces (the spike-gated current, the LIF step, the trigger set, the clock, the fused output) is re-derived on every tick from the previous state by a separately written path.
- A recorded AI-assisted review and an eighteen-mutant campaign, reported in Chapter 7. Its selected cases motivated new checks for pipeline composition, gate behavior, parser handling

and buffer bounds. The final implementation is supplied; the original eighteen-mutant harness and earlier archive are not part of this package.

- A validation-method package containing a mutation engine, a registry of forty-six claims, ten generative properties and an evidence index. The current ledger supports 437/446 killed mutants, with nine survivors assigned verdicts. Those labels record decisions within the campaign scope; they are not a certificate that no defects remain.
- A proposed scaling roadmap, including conditional dispatch arithmetic and explicit requirements for profiling, batching and comparison with a direct classical baseline.

1.5 Document Structure

Chapter 2 situates the system against the graph-neural-network, spiking-neural-network, quantum-graph-neural-network, WebAssembly, and mutation-testing literatures it draws on, and documents the Quantinuum precedent for its central architectural analogy. Chapter 3 describes the architecture layer by layer. Chapter 4 states the validation methodology in its five modes. Chapter 5 covers implementation and the toolchain constraints that shaped it. Chapter 6 reports the empirical results of the suite as it now stands. Chapter 7 reports the independent review: what it reproduced, what it re-derived, what it injected and what survived, what it probed, what it changed, and what in the first revision it corrected. Chapter 8 reports the systematized method: the mutation engine, the claim registry and build index, the generative harness, the baseline and final campaigns, and what the mechanism found that the review had not. Chapter 9 discusses what the results do and do not establish. Chapter 10 proposes a staged path toward the scale that motivated the project. Chapter 11 states limitations without hedging. Chapter 12 concludes.

2. Background and Related Work

2.1 Graph Neural Networks

The graph neural network, in its original formulation [1], propagates node states through a recursive neighborhood-aggregation function until the network reaches a fixed point. Kipf and Welling's graph convolutional network [2] replaced that recursive formulation with a fixed number of spectrally-motivated linear propagation layers, which is the form in which most present-day GNN layers are actually written. Hamilton, Ying, and Leskovec's GraphSAGE [3] is the more direct ancestor of the layer implemented in this project: an inductive, sampling-based neighbor-aggregation rule of the shape h' = σ(h W_self + mean_neighbor(h) W_neigh + b), which Section 3.4 shows is exactly the update rule gnn/message_passing.py implements. PyTorch Geometric [4] is cited not for its model architectures but for its systems answer to a question this thesis's own scaling chapter has to answer as well: how sparse message-passing is actually executed efficiently, as a small number of gather/scatter primitives rather than as operations on a dense adjacency matrix. Section 3.1 adopts that same discipline from the first line of code, and Section 10.1 proposes adopting the same sparse-matrix scatter-sum technique PyTorch Geometric uses internally as the next optimization once this project moves past toy scale.

2.2 Spiking Neural Networks and Neuromorphic Computing

Spiking models motivate explicit temporal state and event-based communication [5, 6]. This scaffold uses a forward-Euler LIF step and gates outgoing messages using the previous tick's spike mask. Loihi [7] provides a hardware precedent for spike-driven computing, but its efficiency does not transfer automatically to this software. Here every neuron is stepped on every tick; reduced circuit calls do not eliminate that node-count-dependent work. No energy advantage is measured.

2.3 Quantum Graph Neural Networks and Variational Quantum Circuits

Verdon et al. [8] study graph-structured parameterized quantum models, and Cerezo et al. [9] review variational quantum algorithms. This scaffold draws inspiration from that work but uses an untrained two-qubit reference map chosen for analytical tractability. Its angles come from a node's existing embedding. Neighbor messages affect spike timing rather than directly becoming a circuit angle. The implementation is not claimed to reproduce a trained QGNN or to represent the full practical regime of quantum algorithms.

2.4 OpenQASM and WebAssembly: Two Assembly Languages, Two Roles

OpenQASM 2.0 [10] supplies the source-language notation for the restricted gate simulator, while WebAssembly [12] supplies the bytecode target for the classical C core. The analogy is an interface pattern that separates loading an artifact from invoking its operation. It does not establish identical execution semantics, universal portability, deterministic outputs or low invocation cost. Those properties depend on each implementation and its execution environment.

2.5 Precedent: Real-Time Classical Control Co-Located with Quantum Hardware

Quantinuum's official QEC-toolkit announcement and examples [11] establish a precedent for Wasm-based classical processing during quantum execution. That is relevant architectural context. The present scaffold does not reproduce Quantinuum's calling direction, memory interface, hardware integration or performance. Earlier precise quotations about WAVM startup and network behavior are omitted because an accessible source establishing those exact statements was not recovered in this review.

The present system does not access Quantinuum hardware or WAVM. It uses a host-orchestrated graph tick with a Wasm LIF step and a separately invoked simulator. The relationship to a QPU-driven decoder is an analogy in separation of responsibilities, not identity of call direction, memory interface or timing guarantees.

2.6 Distributed Graph Systems at Scale

DistDGL [14] implements and evaluates distributed graph training, partitioning and communication. It is a possible substrate to study for future work. Its results do not establish the scalability of this scaffold, and graph partitioning quality, communication overhead and workload locality still require measurements for the proposed use case.

2.7 Mutation Testing

Mutation testing changes a program in specified ways and observes whether tests detect the changes. Early work includes DeMillo, Lipton and Sayward [16]; Jia and Harman [17] survey its development and limitations. A score is relative to the generated mutants and the outcome policy. Some surviving mutants are behaviorally equivalent within a stated domain; others reveal missing tests, accepted risks or unresolved questions. Classifying them can require analysis beyond the campaign itself.

3. System Architecture

3.1 Design Principles

The graph is represented sparsely. For context, a dense float32 adjacency matrix for 2x10^10 nodes would contain 4x10^20 entries and require 1.6x10^21 bytes, before other state. This arithmetic motivates sparse representation but does not establish a feasible sparse deployment. Separately derived oracles and explicit scope statements guide the implementation and its review.

3.2 The QGNN Layer: From-Scratch Statevector Simulation

The parser supports one quantum register, a classical register declaration, h, x, y, z, s, t, rx, ry, rz, cx, barrier and measurement syntax. Gate evolution uses floating-point statevectors. load() parses the artifact and checks specified structural conditions; call() binds parameters and evaluates it. This is a restricted OpenQASM 2.0-inspired execution contract, not a conformant implementation of arbitrary OpenQASM programs: measurement placement, collapse and classical-bit mapping are not modeled. Invocation cost is not benchmarked. The finite-number validation caveat in Section 11 also applies.

QasmResult contains expectation_z, computed from the ideal statevector without shot noise, and sampled_expectation_z, estimated from simulated shot counts. Both are produced by classical computation; the latter is reproducible when the pseudorandom stream and inputs are fixed. Distinct field names document the contract but do not make incorrect caller use impossible.

The circuit ry_node_update.qasm is ry(a) q[0]; ry(b) q[1]; cx q[0],q[1]; rz(phi) q[1], acting on |00>. In the composed pipeline described in Table 1, a, b and phi are the node's pre-tick embedding components 0, 1 and 2. The neighbor aggregate affects LIF current and trigger timing; it is not directly passed as b. CX can entangle the qubits for suitable inputs, but does not do so for every angle pair. The consumed Z expectations have the following closed form:

$$\langle Z0 \rangle = \cos(a) \qquad \langle Z1 \rangle = \cos(a)\cdot\cos(b)$$

The terminal RZ(phi) commutes with the Z observables and changes no final computational-basis probabilities, so phi does not affect either displayed expectation. The consumed values can be evaluated directly by a classical cosine calculation, which is the first performance baseline for this primitive. The final gate-evolution test suite has been rerun successfully. The earlier review also reports a separate full-statevector comparison at 443 parameter points; its original standalone review script is not in the supplied package.

3.3 The SNN Layer: Leaky Integrate-and-Fire as an Activity Gate

snn/lif_neuron.py implements the standard leaky integrate-and-fire update, forward-Euler discretized:

$$V[t+1] = V[t] + (dt/\tau)\cdot(-(V[t]-v_{rest}) + R\cdot I[t]), \quad \text{spike if } V[t+1] \geq v_{th}, \text{ then } V[t+1] = v_{reset}$$

For constant current, define V_inf = v_rest + R I and r = 1 - dt/tau. Before a reset, V_k = V_inf + (V_0 - V_inf) r^k. In the monotone suprathreshold regime 0 < dt/tau < 1 and v_reset < v_th < V_inf, the exact-arithmetic first-spike step from reset is ceil(log((V_inf-v_th)/(V_inf-v_reset))/log(r)). The continuous-time interspike interval is tau log((V_inf-v_reset)/(V_inf-v_th)); its reciprocal is not exactly the discrete rate 1/(k dt) at finite dt. Other parameter regimes and floating-point threshold boundaries require separately specified handling.

The Euler update is vectorized across all nodes. A nonspiking node contributes no outgoing gated message and triggers no circuit evaluation, but the current implementation still steps every neuron and produces per-node outputs each tick. Total cost therefore retains at least a linear component in node count. For constant current, the discrete closed form and continuous firing-rate formula are separate oracles; their parameter domains and discretization relationship must be stated. The >= firing convention is reported to be tested explicitly at threshold in both engines.

3.4 The GNN Layer: Sparse Message Passing

gnn/message_passing.py represents a graph as a directed edge list (Graph: num_nodes, edge_src, edge_dst) and implements neighbor aggregation as a gather followed by a scatter-sum: node features are gathered along edge_src, then accumulated onto edge_dst via NumPy's np.add.at rather than the more obvious out[index] += messages, because the latter silently drops repeated indices under fancy-index assignment — a genuine, well-known NumPy correctness gap, not a stylistic choice, and one this project regression-tests directly (Section 6.1) and the review confirmed by mutation (Section 7.3, M5). gnn_layer_forward implements one GraphSAGE-style layer, h' = σ(h W_self + mean_neighbor(h) W_neigh + b), following Hamilton, Ying, and Leskovec's formulation [3] with fixed, untrained weights: the deliverable is a verified-correct architecture, not a trained model (Chapter 11). gated_mean_aggregate is the variant run_tick actually uses, restricting aggregation to edges whose source node spiked on the immediately preceding tick — the literal wiring point between the SNN and GNN layers described in Section 3.6. A precision the first revision did not state: the composed pipeline uses only this gated aggregation primitive and a feature-mean readout. gnn_layer_forward — the weighted, tanh-activated GraphSAGE layer — is validated in isolation but does not run inside the tick, so the "GNN" in the composition is the spike-gated neighbor mean, not the learned-layer transform.

3.5 The Fusion Layer: An Honest Four-Dimensional Shared Space

fusion/shared_space.py first projects an embedding into three coordinates using a fixed Gaussian matrix P with entries of variance 1/3. For fixed x, E[||Px||^2]=||x||^2 over independent P. For nonzero x, the ratio ||Px||^2/||x||^2 has distribution chi-square(3)/3 and relative standard deviation sqrt(2/3), about 81.65%, for a single projection. Averaging thousands of independently drawn matrices can verify the expectation identity without establishing small distortion for the single matrix used in the pipeline. The fourth field is elapsed simulation time since the last spike, in dt units. If never-spiked nodes use +infinity, the result is an embedding-plus-age record with an extended-real sentinel rather than an ordinary point in R^4; indexing and serialization must specify that case.

maybe_trigger_qgnn receives the current spike mask. For each spiking node it supplies the pre-tick embedding components (0,1,2) as (a,b,phi), then blends the exact values cos(a) and cos(a)cos(b) into components 0 and 1. Component 2 is retained as an interface convention because the present terminal Z readout is insensitive to phi. The neighbor aggregate influences the current and trigger schedule, not b directly. The manuscript reports that the sampling seed now changes with tick; seed collision bounds and stream-independence claims require separate validation. Reusing a seed would reuse pseudorandom variates, but changed circuit probabilities need not produce identical sampled outcomes.

3.6 The Orchestrator: Composing the Layers Into One Tick

orchestrator/pipeline.py runs the five-step sequence summarized in Table 1 once per tick. Two ordering details are load-bearing: the QGNN call in step 4 reads the embedding as it stood entering the tick, before any step in the same tick has modified it; and a node's outgoing message in step 1 exists only on ticks it spiked, filtered by the exact previous-tick spike mask, not by any smoothed or time-averaged notion of recent activity. The first revision said both details were "checked directly, not merely asserted." The review found that to be true of the first (a wrong value fed to the circuit is caught) and false of the second (the composition test never examined the current at all): a pipeline that ignored the spike mask entirely in step 1 passed every one of the 648 assertions then in place. Table 1's last column now records, per step, what the original suite checked in the composition and what the review added.

Step	Layers involved	Operation	How it is checked (Sec. 6, Sec. 7)
1	SNN → GNN	gated_mean_aggregate: only nodes that spiked on the previous tick contribute a message to their out-neighbors	Boundary identities and naive-loop agreement in isolation (6.1); since the revision, re-derived per tick by an explicit per-edge loop (7.3)
2	GNN → SNN	Aggregated neighbor signal reduced to a scalar (feature mean) and added to a fixed per-node baseline current	Not checked in the composition by the original suite (mutants M14, M15, M18 survived); re-derived per tick since (7.3)
3	SNN core	Every neuron stepped one dt via a pluggable LifEngine (Python or wasm32)	Numerically equal agreement between engines, incl. under feedback (6.2); re-derived per tick by a scalar loop (7.3)
4	SNN → QGNN → GNN	maybe_trigger_qgnn: nodes that spiked THIS tick have ry_node_update.qasm called with their PRE-tick embedding as (a,b,ϕ); $\langle Z0\rangle$,$\langle Z1\rangle$ fold back into the embedding	Every trigger's value re-derived from the closed form (6.3); trigger SET checked against the spike mask only since the revision (M2, M3 survived before; 7.3)
5	Fusion	fuse_to_4d projects the embedding to R^3 and appends elapsed time since last spike	Exact concatenation in isolation (6.1); output re-derived per tick from the post-update embedding and the spike log since the revision (M1, M13, M17 survived before; 7.3)

Table 1. The five-step per-tick pipeline (orchestrator/pipeline.py::run_tick), how each step was checked in the composition by the original suite, and what the review's per-tick re-derivation added. Mutant identifiers refer to Table 3.

The LifEngine used in step 3 is a Protocol-typed seam with two interchangeable implementations — python_lif_engine, in-process, and wasm_bridge.py's wasm_lif_engine, backed by orchestrator_core.wasm — selected by the caller and proven interchangeable in Section 6.2, including under the live feedback loop this section describes. Since the revision, a test also confirms that wasm_lif_engine actually spawns the Node harness once per call: the review noted that nothing had asserted this, so a bridge that quietly fell back to the Python step would have "agreed" with it perfectly (Section 7.3, M16).

3.7 wasm-core: The Classical Control Plane, Realized

The C core implements the LIF update without libc or libm and targets wasm32 through clang's freestanding backend. State arrays have a static capacity of 65,536 neurons. Exported pointer functions let the Node host access arrays within Wasm linear memory. This describes the supplied implementation; it is not evidence that an external quantum platform uses the same ABI. The package includes a 560-byte WASM binary whose identity is recorded in the release manifest.

The review added one guard and one distinction. The guard: orc_tick now returns −1 and does nothing if asked to step more neurons than its buffers hold. The distinction: WASM's sandbox protects the host from the module, not the module's own buffers from each other. Called directly with n = 65,538, the original core silently advanced I[0] and I[1] as if they were neurons — V[] and I[] are adjacent in linear memory — and returned normally; only a far larger n reached the end of linear memory and trapped (Section 7.4). The Node harness had always refused over-capacity requests before calling in, so the suite never saw this; the point is that "sandboxed" is a property of the host boundary, and a module that is to be the always-on control plane of a real system must be safe on its own.

The C-to-wasm32 LIF core realizes the proposed separation between classical state update and circuit evaluation. The reported tests compare its numerical outputs with Python in isolated, feedback-coupled and full-capacity cases. The current integration launches a Node subprocess for each tick, so persistence, low-latency invocation, near-native performance and real-time deadlines remain unmeasured rather than established properties of the delivered system.

4. Methodology: Adversarial Validation

4.1 Why Adversarial Validation

A system combining three computational paradigms that are individually easy to get subtly wrong — discrete-time numerical integration, sparse graph aggregation, and quantum circuit simulation — cannot be trusted on the strength of its own internal consistency: code that is wrong in a self-consistent way produces output that looks exactly as plausible as code that is right. Every result reported in Chapter 6 is therefore checked against something derived independently of the implementation under test, and the discipline is not merely stated as a principle in this chapter; it altered the actual content of this thesis while it was being written. An early version of the full-pipeline experiment behind Section 6.3 used a simulated time window short enough, relative to the neuron time constant, that no node ever crossed its firing threshold — a scenario that would have silently validated nothing. The check requiring "at least one spike occurred" is what caught this, immediately and automatically, and the corrected parameters are what Section 6.3 reports.

The revision adds a second episode of the same kind, at a higher level. The first revision of this document claimed, in its account of Section 6.3, that the composition test "is what makes it capable of catching" a spike attributed to the wrong node. The review tested that sentence by attributing spikes to the wrong node — and the test passed. The sentence was false, for a reason the next section explains, and the code, the suite, and the sentence were all changed. Both episodes are included here as evidence for this chapter's claim, not as illustrations of it: adversarial validation catches real defects, and it catches them in the validators too.

4.2 Five Validation Modes

Exact closed-form comparison

Closed-form comparison evaluates an implementation against a separately derived formula with an explicit domain and tolerance. It can expose errors that two similar implementations share. The derivation, its transcription and the comparison code can themselves be wrong, so this is a strong check within its assumptions rather than an absolute source of truth.

Independent cross-implementation comparison

Cross-implementation comparison uses differently implemented numerical paths: a naive edge loop versus vectorized aggregation, C/Wasm versus Python LIF arithmetic, and dense Kronecker operations versus stride-based gate evolution. Agreement supports the exercised cases but can miss shared conceptual errors. Independence must be stated component by component; the quantum references share parts of the production frontend.

The supplied gate-evolution references use independently written dense matrix operations but share the production parsed-circuit representation and parameter evaluator. Their independence therefore applies to gate evolution, not parsing or parameter-expression semantics. P9 likewise reuses the

production LIF step for its expected voltage/spike output; the dedicated composition-wiring test supplies a separately written scalar path.

Statistical validation

Used only where the quantity under test is inherently statistical — shot-sampled quantum measurement, or the Gaussian squared-norm unbiasedness property in Section 3.5 — and always reported as statistical, with the observed value, the observed error, and the tolerance shown together, rather than presented with the same unqualified confidence as an exact check. The review added a calibration the first revision lacked: the suite's 2.088% Gaussian squared-norm mean error at 4,000 trials is about 1.6 standard deviations of the estimator (whose relative standard deviation at that trial count is 1.29%), so it is unremarkable rather than a near miss; and the 10% tolerance is loose — nearly eight standard deviations — which means the check would catch a missing 1/√3 factor (a 200% error) but not a subtle miscalibration.

Generative (property-based) testing (added in the third revision)

A generative test applies a closed-form or cross-implementation oracle to samples from a specified input generator. Seeded sampling improves repeatability and broadens the exercised cases, but the generator, its distribution and its limits remain design choices. Passing finitely many draws is evidence for those draws, not a proof over the full input domain.

Mutation testing (added in the second revision; made systematic in the third)

Mutation testing asks whether the suite detects specified changes to the implementation. In the supplied runner, a nonzero test-process exit can kill a mutant, whether caused by an assertion failure or a crash. A timeout is classified separately. A survivor needs investigation: it may expose a coverage gap, be equivalent within the supported domain, or represent an accepted or unresolved risk. Neither the registry nor the generated index turns those judgments into universal proof.

4.3 The Discipline Applied at Each Layer

Every numerical function added to this project during its construction was paired with one of the first three checks above before being considered complete, and in several cases a genuinely independent closed form had to be derived specifically for the test rather than reused from the implementation's own analytic formula — most notably in the SNN layer, where the continuous-time steady-state firing rate (Section 3.3) is a different, weaker claim than the exact discrete-time spike-step formula actually used to check the implementation, derived separately in closed form from the Euler recurrence itself so that every individual spike, not merely an average rate, could be checked to near machine precision. What the original construction did not do is apply the last check to itself. The composition test in particular was written to re-derive each triggered node's new value from the closed form — and did so correctly — but took the set of triggered nodes, and every other per-tick quantity, from the pipeline's own log. It was, without anyone noticing, checking the pipeline against the pipeline for everything except the one arithmetic step it re-derived. Chapter 6 reports the resulting counts as they now stand; Chapter 7 reports how that gap was found and closed.

5. Implementation

5.1 Toolchain and Constraints

Two dependency choices in this project were not preferences; they were forced, and the forcing shaped the architecture in ways worth recording rather than presenting as if they had been freely chosen. pip install qiskit and pip install wasmtime both failed in the sandbox this project was built in, with an error consistent with a restricted package index rather than a hard network block — plain installs of other packages, including NumPy, succeeded. Rather than leave that ambiguous or block on it, qmodule's statevector simulator was built from scratch. This is presented in Chapter 11 as a limitation on gate coverage and scale, not as a discovery that a bespoke simulator is preferable to Qiskit in general — it is not — but the resulting simulator is self-contained, auditable line by line, and free of any external quantum SDK's own version and licensing surface, which is a real advantage for a scaffold meant to be read and extended by a second, independent coding agent. The review exercised exactly that advantage: it validated the simulator against an independent reference on the full complex statevector, something that would have been considerably harder to do meaningfully against an SDK whose internals the reviewer did not own.

The second constraint was sharper: rustup target add wasm32-unknown-unknown failed with an explicit HTTP 403 organizational policy denial when attempting to reach static.rust-lang.org. Per that proxy's own stated policy, an explicit 403 is a real refusal to be reported, not a transient failure to be retried. The resolution — clang's own built-in wasm32-unknown-unknown backend, which requires no additional toolchain download at all — was verified end-to-end (compiled a trivial function, loaded and called it from Node's native WebAssembly API, confirmed the correct result) before being adopted for wasm-core/. Neither constraint is necessarily true of every environment; both are recorded in AGENTS.md so that whichever environment continues this work does not have to rediscover them by repeating the same failed installs.

5.2 Code Organization

The Python packages separate circuit simulation, spiking dynamics, graph aggregation, fusion and orchestration. The wasm-core directory contains the C kernel, build script and Node harness. Module docstrings and the claim registry connect design choices to tests; repository history and the source manifest make later changes traceable.

Section 7.5 records the earlier review's code and test changes. Revision 4 preserves the supplied implementation while correcting its explanatory claims and adding a current reproduction record.

6. Empirical Results

6.1 Layer-by-Layer Validation Results

Table 2 summarizes the state of the full test suite after the second and third revisions, runnable in one command (bash run_tests.sh) from a clean checkout. Every number below was produced by an actual run of that command from a clean extraction of the delivered archive (hybrid-gnn-snn-qgnn-scaffold-v1.2.zip), not estimated. Of the five original modules, two are unchanged; test_pipeline_smoke.py had one assertion added and two tightened from a 10^{-9} tolerance to exact equality in the second revision (Section 6.2), and test_qasm_sim.py and test_gnn_message_passing.py gained checks in the third for gaps the mutation campaign exposed (Section 8.5). Beyond the suite, the tree now carries a mutation score — 437 of 446 generated mutants killed, every survivor with a verdict — and a registry of forty-six claims; both are reported in Chapter 8 and reproduced in the repository's BUILD_INDEX.md.

Test module	Checks	Result	Primary validation mode	Origin
test_qasm_sim.py	25	ALL PASSED	Exact closed-form	original +5 (rev. 3)
test_qasm_gate_coverage.py	30	ALL PASSED	Cross-implementation (full statevector)	review +2 (rev. 3)
test_qasm_parser_robustness.py	37	ALL PASSED	Specified rejection of malformed input	review +7 (rev. 3)
test_lif_neuron.py	306	ALL PASSED	Exact closed-form (discrete)	original
test_gnn_message_passing.py	52	ALL PASSED	Cross-implementation + exact identities	original +11 (rev. 3)
test_shared_space.py	24	ALL PASSED	Exact closed-form + statistical	original
test_pipeline_smoke.py	649	ALL PASSED	Cross-implementation + closed-form	original (+1, 2 tightened)
test_composition_wiring.py	135	ALL PASSED	Per-tick independent re-derivation	review
test_wasm_core_bounds.py	18	ALL PASSED	Direct calls at and beyond capacity	review +5 (rev. 3)
test_properties.py	10 props.	ALL PASSED	Generative, over random inputs	rev. 3
Total	1,286	0 failures		1,039 original

Table 2. Manuscript-reported results across ten test modules: 1,286 passing check entries and zero reported failures. The count includes one entry per property; the ten properties cover 2,350 random trials. These are printed check entries rather than 1,286 independent properties or necessarily 1,286 distinct assertions.

A note on what the count measures, added by the review. It is a count of printed pass lines, and it is dominated by repeated instances of a few check types: 300 of test_pipeline_smoke.py's checks are the per-tick shape check on the fused output, 333 are the three per-trigger closed-form checks over 111

triggers, and 210 of test_lif_neuron.py's are per-spike time comparisons. The number of distinct properties checked is forty-six, by the registry of Section 8.2, across 54 test functions. Both numbers are honest; the second is the better measure of coverage, and neither is the measure that matters most — Sections 7.3 and 8.5 report the one that does.

The fixed node-relabeling test uses a 30-node graph with 180 edges and checks the nodewise output after undoing the permutation, with an allowed maximum absolute difference of 10^-10. This tests equivariance on that case. The projection check averages 4,000 independent draws and reports 2.088% relative error against a 10% tolerance; Section 4.2 explains the sampling uncertainty. The gate-coverage module compares supported gate evolution with a dense reference, including phase-sensitive circuits and qubit-order cases. These suite checks were rerun; their finite scope and shared frontend must be retained when interpreting them.

The supplied relabeling tests check permutation equivariance with maximum absolute error at most 10^-10: one fixed 30-node graph test and P6's 200 generated cases. This numerical threshold is a test criterion, not an observed universal error bound or proof of bit equality. In exact arithmetic, shared weights, rowwise activation and consistently relabeled neighbor means give F(PAP^T,PH)=P F(A,H).

6.2 Case Study: Zero-Tolerance Cross-Language Agreement

The original experiment record compares the C/WASM and Python LIF implementations over 300 ticks and 25 neurons. It reports zero maximum absolute difference across 7,500 membrane-potential values, matching spike flags and 37 spikes in each engine. The test now asserts zero-tolerance numerical agreement. Revision 4 reran the supplied test module successfully. These comparisons establish equality for the exercised numeric values, not all binary64 encodings or every compiler/runtime environment.

That result alone establishes only that two implementations agree on isolated, externally-supplied input. A separate, harder experiment re-ran the same comparison with the input current itself produced fresh, every tick, by the previous tick's spikes propagating through the graph-aggregation layer — a live feedback loop, not a fixed scenario. Across 25 feedback-coupled ticks on a 10-node graph (38 total spikes), the maximum observed disagreement between the Python and WASM engines remained exactly 0.0 for both membrane potential and downstream embedding state, with zero spike-mask mismatches, now also asserted at zero tolerance. The review extended the result in two directions: at the module's full static capacity of 65,536 neurons, 20 ticks through the real harness — 1,310,720 neuron-ticks, 19,264 spikes — agree with the Python reference to 0.0 with no flag mismatches; and at an input constructed to land exactly on the firing threshold (dt/τ = 1/2, V = 0, I = 2, so V[t+1] = 1.0 = v_th to the bit), both engines spike and both reset, and inputs landing 5×10^{-7} below and above the threshold do not and do, respectively, in both engines.

Zero numerical difference and zero-tolerance equality do not establish binary64 byte identity: positive and negative zero compare equal but have different encodings, and JSON.stringify does not preserve negative zero or non-finite values as their original bit patterns. The reported result should therefore be read as exact numerical equality for the tested domain. A bitwise claim would require raw-byte comparisons, a signed-zero and non-finite-value policy, and pinned compiler flags, instruction features

and runtime versions. Matching arithmetic order and round-trippable finite decimal values can explain the observations but do not establish every portability claim.

6.3 Case Study: Re-Deriving 111 Live Quantum Triggers From Closed-Form Math

A 14-node toy graph — a directed ring plus random chord edges, constructed so every node has at least one incoming edge — was run through 300 ticks of the full five-step pipeline described in Section 3.6, with a baseline input current chosen to guarantee spontaneous spiking within the simulated time window. Across that run, 111 individual quantum-graph node updates were triggered. For every one of them, this project did not merely check that some update occurred: it re-derived, from the closed form proven in Section 3.2, exactly what the resulting embedding values 0 and 1 had to equal given that specific node's specific pre-tick embedding values 0 and 1 as (a, b) — and confirmed all 111 to a tolerance of 10^{-9}. A further 4,089 node-ticks on which no trigger occurred were confirmed to leave that node's embedding numerically unchanged.

What this result establishes must be stated more carefully than the first revision stated it. It establishes that whenever the pipeline reports a trigger, the value written is the correct function of the pre-tick state, and that nothing else was written: a result written into the wrong node's row, an embedding fed to the circuit after some other step had modified it, or a wrong closed form would each have produced a mismatch at exactly the affected nodes and ticks (mutants M4 and M9, both killed). It does not establish that the pipeline reports the right triggers. The test takes the set of triggered nodes on each tick from the pipeline's own log and never compares that set to the spike mask, so a trigger fired one tick late, or on the neighboring node, produces values that are still the correct closed form of their own pre-tick state — and passes (mutants M2 and M3, both survived). The same structure leaves the spike-gated current, the LIF step inside the loop, the elapsed-time clock, and the fused output entirely unexamined in the composition (M1, M13, M14, M15, M17, M18, all survived). Section 7.3 gives the full table; test_composition_wiring.py now re-derives all of these on every tick from the previous state, and kills all of them.

6.4 Known Gaps

The suite is finite. It exercises the supported gate subset and selected malformed inputs, small composed graphs, and a separate full-capacity LIF comparison. It does not execute the weighted GraphSAGE layer inside the composed tick, general OpenQASM measurement semantics, quantum hardware, trained models or distributed execution. The simulations are sequential on one machine, with Node child processes used for Wasm calls; the mutation runner can use multiple workers. Shared frontend code limits the reference simulator's independence. Current source-level caveats are consolidated in Section 11.

7. Historical AI-Assisted Review

7.1 Brief and Method

This chapter preserves the substance of the earlier AI-assisted review as a historical account, with factual corrections made explicit. The supplied final package contains the resulting tests and source but not the earlier archive or the original eighteen-mutant harness. Revision 4 reruns the final suite and inspects its actual oracles; it does not represent that rerun as a reproduction of every earlier experimental step.

7.2 Reproduction and Independent Re-Derivation

The earlier review reports reproduction of 1,039 check entries, 7,500 matching membrane potentials, 111 circuit-trigger updates, 4,089 unchanged node-ticks and the recorded projection statistic. Those claims describe that earlier run. The present package supports a fresh run of the final ten-module suite and inspection of the final campaign ledger. Earlier archive identity, intermediate results and precise historical documentation quotations cannot all be reconstructed from the final tree alone.

The re-derivations went further than the suite had. A reference simulator that builds the full 2^n × 2^n unitary of each gate with Kronecker products — a different algorithm from the scaffold's stride-based in-place update, and one that makes endianness and CX orientation explicit — agrees with the scaffold on the full complex statevector of ry_node_update.qasm at 443 parameter points, to a maximum amplitude difference of exactly 0.0, and independently confirms $\langle Z0 \rangle = \cos(a)$ and $\langle Z1 \rangle = \cos(a)\cos(b)$ to within 7×10^{-16} at all of them. The same reference agrees with the scaffold on each of the six gates the first revision listed as untested, alone and inside an h-G-h sandwich (which makes phase gates visible to a Z measurement, with hand-derived closed forms: h z h gives $\langle Z \rangle = -1$, h s h gives 0, h t h gives $\cos(\pi/4)$), on rx and ry swept over two full periods against $\langle Z \rangle = \cos\theta$, on a three-qubit GHZ state, and on a CX whose control index exceeds its target's. The discrete LIF closed form $V_k = v_{inf} + (v_{reset} - v_{inf})(1 - dt/\tau)^k$ was re-solved and matches the Python step to 6×10^{-15} along the whole pre-spike trajectory at eight currents, with the first-spike step matching the ceiling formula exactly. The Gaussian squared-norm identity was re-estimated at 200,000 trials: 0.077% relative error, against an estimator standard deviation of 0.18%.

Two of these deserve comment. The gates the first revision called untested are correct — the gap was in the evidence, not the code. And rz had, until the review, no positive validation at all: the suite's only rz check was that it does not move $\langle Z1 \rangle$, which an rz implemented as the identity also passes (Section 7.3, M10). The reference simulator's h; rz(θ); h check, with closed form $\langle Z \rangle = \cos\theta$ over 41 angles, is the first test in the project that would fail if rz did nothing.

7.3 Mutation Testing: The Suite's Claims Tested

Eighteen defects were chosen, each corresponding to a specific claim in the first revision about what the suite would catch, or to a specific place where the reviewer suspected it would not. Each was injected on its own into a fresh copy of the tree — a single-line change in every case — and the relevant test modules run; a mutant that produced at least one failure was recorded as killed, one that passed unchanged as

survived, following the standard terminology [16] [17]. Table 3 gives the outcome against the suite as delivered and against the suite with the four review modules added.

#	Injected defect	Where	Original suite	With review modules
M1	Fused output computed from the pre-update embedding (steps 4/5 swapped)	pipeline	survived	killed
M2	QGNN triggered from the previous tick's spike mask (one tick late)	pipeline	survived	killed
M3	Spike mask rolled by one node before QGNN (neighbor triggered)	pipeline	survived	killed
M4	QGNN result written into the wrong node's embedding row	fusion	killed (119)	killed
M5	out[index] += messages in place of np.add.at	gnn	killed (23)	killed
M6	Random projection without the 1/√3 scaling	fusion	killed (1)	killed
M7	LIF threshold ≥ changed to >	snn	survived	killed
M8	Post-spike reset removed	snn	killed (56)	killed
M9	ry gate applies half the requested angle	qmodule	killed (12)	killed
M10	rz gate replaced by the identity	qmodule	survived	killed
M11	C core computes the update in single precision	wasm-core	killed (2)	killed
M12	Aggregation gated on the destination node's spike, not the source's	gnn	killed (1, unit test only)	killed
M13	Elapsed-time clock off by one tick	pipeline	survived	killed
M14	Spike gating dropped from the pipeline's aggregation (every node always sends)	pipeline	survived	killed
M15	GNN → SNN coupling sign flipped	pipeline	survived	killed
M16	wasm_lif_engine silently runs the Python step instead of the module	bridge	survived	killed
M17	last_spike_time updated from the previous tick's mask	pipeline	survived	killed
M18	Readout sums over features instead of averaging	pipeline	survived	killed

Table 3. Mutation results. Parenthesized numbers are how many assertions the original suite failed. The suite as delivered killed 7 of 18; with the review's modules it kills 18 of 18.

Table 3 shows that all eight selected pipeline-composition mutants survived the original suite, while seven other selected mutants were killed. Three additional survivors concerned a threshold boundary, rz behavior and actual backend invocation. M14 illustrates the composition gap: the pipeline could ignore its spike gate yet still pass a test that re-derived values only for the triggers reported by that same pipeline. This supports more independent wiring checks; it does not show that the original suite caught every value-level defect.

The other three reported survivors expose distinct checks: M10 requires a phase-sensitive test so RZ cannot be replaced silently by the identity; M7 requires an explicitly constructed threshold-boundary input; and M16 requires confirming that the intended backend was actually invoked. Random continuous inputs need not land on an exact threshold, and output agreement alone cannot distinguish a genuine cross-backend comparison from two calls to the same implementation.

The four review modules were written to kill the survivors by the project's own rules — an independent, deliberately naive re-derivation, compared exactly — rather than by asserting the specific mutations away. test_composition_wiring.py re-derives, on every one of 120 ticks, the current from a per-edge loop over the previous tick's spike mask, the LIF step from a scalar loop, the trigger set from the spike mask, the last-spike clock from the spike log alone, and the fused output from the post-update embedding by a triple loop; it also constructs the exact-threshold input and counts subprocess invocations. test_qasm_gate_coverage.py is the reference simulator of Section 7.2, made permanent. With them in place, every mutant dies, and the mutation harness re-run against the fixed tree confirms 18 of 18.

7.4 Probing the Admitted Gaps

Section 6.4 of the first revision said the parser's behavior on malformed input was "unspecified, not merely untested," and that nothing exercised the WASM engine near its capacity. Both statements were accurate; the review made them specific. Table 4 records what each probe actually did before the revision and what it does now.

Probe	First-revision behavior (observed)	Now
Unknown gate name	load() succeeds; call() raises NotImplementedError	SyntaxError at load
Qubit index beyond the register	load() succeeds; call() raises ValueError: negative shift count	SyntaxError at load
cx q[0],q[0]	Accepted; silent no-op (wrong answer, no error)	SyntaxError at load
cx with three operands	Accepted; third operand silently ignored	SyntaxError at load
h(0.3) — parameter on a parameter-free gate	Accepted; parameter silently ignored	SyntaxError at load
Second qreg declaration	Accepted; second silently replaces the first	SyntaxError at load
Gate on an undeclared register	Accepted; applied to the declared register	SyntaxError at load
measure into an undeclared creg	Accepted silently	SyntaxError at load
barrier (valid OpenQASM 2.0)	call() raises NotImplementedError	Accepted as a no-op
Parameter expression 9**9**9**9	call() hangs (killed after 8 s); the whitelist admitted **	ValueError in 0.1 ms
__import__("os") in a parameter	Rejected at parse (unbalanced parentheses)	Rejected at parse

Probe	First-revision behavior (observed)	Now
orc_tick(65,538) called directly	Returns normally; I[0], I[1] silently overwritten	Returns −1; memory untouched
orc_tick(10,000,000) called directly	Traps at the end of linear memory	Returns −1
65,536 neurons × 20 ticks via the harness	Untested	Numerically equal vs Python, 0 flag mismatches
500 random graphs, gated aggregate vs naive loop	Untested (fixed seeds only)	Max difference 0.0
Two identical pipeline runs	Untested	Numerically identical

Table 4. Historical before-and-after observations reported in Revision 3. The final package includes parser, bounds, graph, composition and repeatability checks; exact earlier timings and every original probe are not independently reproduced by a final-suite rerun.

Three of these matter beyond tidiness. The parser accepted several malformed circuits and returned a plausible wrong answer with no error — a cx with identical operands that did nothing, a third operand dropped, a parameter dropped, a register silently substituted for another — which is precisely the failure class this project's methodology exists to prevent. The parameter-expression evaluator was described in its own comment as "deliberately not a general eval() to keep an externally-supplied .qasm file inert," and its character whitelist did exclude names and attribute access; but it admitted the asterisk, and therefore exponentiation, and a ten-character expression made call() hang for as long as the reviewer let it. And the C core, called with two more neurons than it holds, advanced the first two entries of the input-current buffer as if they were membrane potentials and returned a normal spike count — the WASM sandbox, which the first revision leaned on as a property of the design, protected the host and nothing else. Each is fixed in the code and pinned by a test (Section 7.5). The probes that found nothing are recorded too: five hundred random graphs with self-loops, duplicate edges, isolated nodes, empty edge lists and every spiking fraction agree with a naive loop to 0.0; the full pipeline is numerically identical across two runs; and at exactly 65,536 neurons the engine is as exact as at 25.

7.5 What Changed in the Scaffold

The archive delivered with this revision (hybrid-gnn-snn-qgnn-scaffold-v1.1.zip) differs from the first in the following ways, and no others. Four test modules were added — the gate-coverage, parser-robustness, composition-wiring, and wasm-core-bounds modules listed in Table 2, with 28, 30, 135, and 13 checks respectively — and run_tests.sh runs all nine. In test_pipeline_smoke.py, the two WASM-versus-Python assertions were tightened from a 10^{-9} tolerance to exact equality and one assertion on the maximum error was added. In statevector_sim.py, parse_qasm now performs the load-time validation described in Section 3.2 and accepts barrier as a no-op, and the parameter evaluator refuses exponentiation and expressions over 200 characters, and substitutes the constant pi only as a whole word. In orchestrator_core.c, orc_tick returns −1 and does nothing for a neuron count outside its buffers, and the harness treats that return as an error; the module is 560 bytes rather than 553. In pipeline.py, the shot-sampling seed advances with the tick. Three docstrings were corrected to stop referring to files

outside the repository. VALIDATION.md, README.md, AGENTS.md and ARCHITECTURE.md were updated to match, and AGENTS.md gained a testing rule: name the injected defect that would make a new test fail, or call it a smoke test.

Every closed form, every interface, every number reported by the original five modules, and every design decision described in Chapter 3 is unchanged. The review found no numerical claim in the first revision to be false. What it found was a difference between what the suite proved and what the document said it proved, and in three places a difference between what the code was described as doing and what it did on inputs no test had tried.

7.6 Corrections to the First Revision

The earlier revisions corrected the distinction between a value written to the wrong node and a trigger fired for the wrong node, added positive phase-gate tests, tightened numerical-equality checks and added parser and buffer-boundary cases. Revision 4 further corrects the equivariance terminology, the scope of numerical equality, the actual circuit-parameter dataflow, mutation accounting and claims of exhaustive validation. It does not presume that every other historical statement is thereby proved.

8. The Method, Systematized

8.1 From Probe to Measurement

The operator-based campaign extends the hand-selected eighteen-mutant probe by applying a declared set of transformations to eligible source sites. Its score measures detection within that chosen mutation scope, not all possible defects. Both manual and generated campaigns are measurements relative to their selected mutants. The manuscript reports a runner in python/method/ that regenerates an evidence index in about three minutes on two cores; those timing and execution claims await reproduction.

The engine applies six operator families to Python syntax and explicit C source edits: arithmetic, relational, logical and constant replacement, statement deletion and argument swap. Each recorded mutant uses a copied source snapshot. Its test-selection graph and short-circuit order are part of the method, as are compile failures, crashes and timeouts. Defaults are excluded by design; that exclusion limits the fault model and does not show that default or configuration changes can never be defects. Content-derived mutant identifiers support traceability across line-number changes.

Operator	**What it changes**	**Example site in this scaffold**	All-source killed / valid
AOR	arithmetic: + ↔ −, × ↔ ÷, a unary minus dropped	−(V − v_rest) in the LIF update; cos(a)·cos(b)	89 / 92
ROR	relational: ≥ → >, > → ≥, ≤ → <, < → ≤, == ↔ !=	V_next ≥ v_th; q ≥ n_qubits in the parser	38 / 39
LCR	logical: and ↔ or; not x → x	the dtype check in Graph; wasm_available()	18 / 18
CRP	constants: 0 → 1, 1 → 0, c → c + 1	the 1/√3 scaling; 1 << (n − qubit − 1); regex group numbers	124 / 127
SDL	a statement inside a function deleted (replaced by pass)	the post-spike reset; ops.append(MeasureOp(...))	131 / 132
AGS	the last two positional arguments of a call swapped	_apply_cx(state, n, control, target); eval(safe, globals, locals)	37 / 38
C subset (included above)	Twelve textual C mutants, classified under the operator rows above	the update rule's four operators; ≥ → >; the loop bound; the guard; spike_count++	12 / 12 (do not add)

Table 5. Final ledger totals by operator. The first six rows cover all sources and sum to 437 killed / 446 valid. The C row is a nonadditive subset of those rows: its twelve mutants have already been classified under the six operator families. Table 9 independently groups the same ledger by file and reconciles to 437/446. This labeling resolves Revision 3's apparent extra twelve mutants.

One defect the engine found was its own. Its first run read each source file from the live tree at the moment each mutant was applied, and the tree was being edited while the campaign ran; seven mutants were recorded as "site not found" because their spans no longer matched the text. The engine now

snapshots every source at generation time and gives each worker that snapshot. The detail is included because it is the method applied to itself: an "invalid" column that exists to be zero, and was not.

8.2 The Claim Registry and the Build Index

The registry links forty-six stated claims to validation modes and named test functions. That is useful traceability: readers can locate the intended evidence and its scope. Test-name existence and passing status do not by themselves establish that a test semantically verifies every part of its claim. The source review identifies examples where those contracts need further tightening.

The build-index generator combines module structure, suite results, mutation outcomes and survivor verdicts. Its strict mode refuses regeneration when a survivor lacks a recorded verdict. An index is an evidence map, not a proof certificate. Failed or stale regeneration must remain visible beside the last successful index; accepted verdicts and source identity require review.

8.3 Generative Testing Without a Dependency

The seeded property harness runs ten properties over 2,350 input draws and reports reproducible failure information. Its generators choose the tested distributions and bounds. It has no shrinking or automated failure minimization; small generated examples are useful but do not provide that capability. The reported earlier sub-second timing is environment-specific, not a performance promise.

Property	Random inputs	Trials	What it generalizes
P1	valid circuits over the whole supported subset, up to 3 qubits and 8 gates, bound and literal parameters	300	full-statevector agreement with the Kronecker reference (Section 7.2)
P2	malformed lines: 28 known bad forms plus random strings over the QASM alphabet	400	load-time rejection (Section 7.4): SyntaxError or ValueError, nothing else, under a second
P3	(a, b, ϕ) in $[-10, 10]^3$, shot counts in {50, 200, 1000}, random seeds	200	sampled $\langle Z \rangle$ within six standard errors of exact; seed determinism; counts sum to shots
P4	LIF parameters (τ, v_rest, v_reset, v_th, R) and currents, all random	250	the vectorized step, the sub- and supra-threshold closed forms, and the analytic rate — none of which had been checked away from the default parameters
P5	graphs with self-loops, duplicate edges, isolated nodes, empty edge lists; feature scales 10^{-3}–10^{3}; every spiking fraction	400	aggregation against a naive per-edge loop; mass conservation; in-degree
P6	random graphs, random weights, random relabelings	200	permutation equivariance of the GraphSAGE layer
P7	random embeddings and spike times, including never-spiked nodes	300	projection linearity; fused-output concatenation; +inf handling

Property	Random inputs	Trials	What it generalizes
P8	random embeddings in [−10, 10], masks, blends in {0, 0.25, 0.5, 1}	200	the trigger set, the closed form under blending, pass-through of untouched rows
P9	random small pipelines: 1–8 nodes, random graphs, parameters, coupling in [−1, 1.5]	60	every per-tick wiring invariant of Section 7.3, on configurations no one chose
P10	ring graphs driven so every node triggers every tick	40	sampled outputs differ across tested ticks; changing embeddings confound direct seed-distinctness validation

Table 6. The ten generative properties in tests/test_properties.py. Each is registered as a claim and each is the random-input generalization of a fixed check that already existed.

The earlier review reports malformed-expression failures in the first property-harness run, including 1..2 and 1/0, followed by normalization of expression-evaluation errors. The current evaluator checks finite values on its general expression path. Its literal and bound-parameter fast paths bypass that check, so the stronger claim that all accepted parameters are finite is not established by the present source or passing suite.

Source inspection identifies an additional qualification: _eval_param applies math.isfinite only to its eval-based path. Its direct float(expr) and bound-parameter returns bypass that check, so the stronger registry claim that every accepted parameter yields a finite float or raises ValueError is not established. For example, the literal 1e309 converts to positive infinity on that fast path. This is a static source finding, not a newly executed experiment in this review.

8.4 The Baseline Campaign

To measure what the mechanism sees that the reviewer did not, the engine was first run against the suite exactly as revision 2 delivered it — nine modules, 1,246 checks — from a clean extraction of that archive. It generated 464 mutants over eight files and killed 410 of them (88.4%); 47 survived and 7 timed out (Table 7).

File	Mutants	Killed	Survived	Timed out	Score
qmodule/statevector_sim.py	242	210	25	7	86.8%
qmodule/qasm_module.py	21	19	2	0	90.5%
snn/lif_neuron.py	40	38	2	0	95.0%
gnn/message_passing.py	45	34	11	0	75.6%
fusion/shared_space.py	47	46	1	0	97.9%
orchestrator/pipeline.py	39	36	3	0	92.3%
orchestrator/wasm_bridge.py	18	16	2	0	88.9%
wasm-core/orchestrator_core.c	12	11	1	0	91.7%
Total	464	410	47	7	88.4%

Table 7. The baseline campaign against the revision-2 suite (v1.1). The seven timeouts are all in one function.

The manuscript reports 47 survivors and seven timeouts in the revision-2 baseline. The supplied final tests contain the H-RZ-H sweep and full-statevector primitive comparisons described in Chapter 7, as well as the later non-basis-state additions. The assertion that every parameterized gate was previously applied only to |0> is too broad. Without the original revision-2 source and baseline mutant ledger, the eight gate-related survivors cannot be assigned confidently to specific matrix entries, strides or parameter paths. Retain the historical group counts as reported and state this classification limit.

Class of survivor	Count	Example	Why the suite could not see it
Gate-related implementation paths not distinguished by the baseline tests	8	the second column of U(θ, ϕ, λ) mutated; rx with λ = 2π; base + off → base − off in the stride loop	Exact gap unresolved: the blanket basis-state-only explanation conflicts with the H-RZ-H and primitive tests described in Chapter 7; baseline source and mutant diffs required.
Default parameters hide arithmetic	2	v_inf − v_reset → +; −(V − v_rest) → +	every LIF test used v_reset = v_rest = 0
Validator boundaries and connectives	6	< → ≤ on node ids; and → or on dtypes; ≥ → > on measure indices	one out-of-range case and one dtype case each; no boundary case, no mixed case
Redundant guards (equivalent mutants)	10	where(deg > 0, ...) plus mean[deg == 0] = 0; if i < j in CX; × ±1 in ⟨Z⟩	two guards for one condition, or an operation with no observable effect
Unchecked outputs and untested specifications	13	the GraphSAGE layer's bias and W_neigh signs; orc_tick's return value; n_clbits; the sampled-⟨Z⟩ accumulator; renormalization	invariance and boundedness were checked, the value was not; or the documented behavior had no test
A hand-written binary search	7	lo < hi → ≤; mid + 1 → mid − 1; lo = mid + 1 deleted	every mutant is an infinite loop (recorded as a timeout, not a kill)
Conventions and configuration	8	the seed arithmetic (4); a subprocess timeout; a length limit and its boundary; eval's argument order	not defect classes; recorded as accepted or config

Table 8. The 54 baseline survivors and timeouts by class. The first two classes are blind spots of the same shape as the one Chapter 7 found by hand; the mechanism found them without anyone having to imagine them.

Thirteen survivors were outputs that no test had ever compared to a value: the GraphSAGE layer's output had been checked for permutation equivariance, boundedness and shape but never for its value, so the sign of its bias could be flipped; the C core's return value — the spike count — was read by nothing; the parser recorded the classical-register size and nothing checked it; the accumulator for the sampled expectation value could start at 1.0 instead of 0.0 and stay within the statistical tolerance. Seven mutants were timeouts, all in one place: a hand-written binary search inside the shot sampler, where every mutation of the loop bound, the midpoint, or the two update statements is an infinite loop. Ten were equivalent mutants in the strict sense — redundant guards where mutating either of two checks for the same condition changes nothing — and eight were conventions or configuration.

8.5 Refactor and the Final Campaign

Three kinds of change followed, in the order the method prescribes. Six sites were refactored so that an equivalent mutant cannot exist there: mean_aggregate uses one guarded division instead of a substituted divisor plus an overwrite; _apply_cx visits each swapped pair once, from the member whose target bit is clear, so there is no i < j to mutate; the shot sampler uses the standard library's bisect instead of a hand-written search, which removes all seven infinite-loop sites; expectation_z adds ±p instead of multiplying by ±1; the initial amplitude is complex(1.0) rather than 1.0 + 0j; and a redundant strip was removed. Thirty checks were added for the real gaps: parameterized gates applied after h, x, s and t and on entangled three-qubit states; the GraphSAGE layer's value against a naive per-node loop; validator boundaries, negative ids, mixed dtypes and length mismatches; measure indices at the register boundary, the classical-register size, a parameterized circuit called without parameters, the parsed operation list; the sampled expectation recomputed exactly from the returned counts and the sampler's renormalization of an unnormalized state; the C core's return value against the number of flagged neurons; and wasm_available() with the binary absent. Property P4 compares discrete and continuous interspike intervals with an explicit discretization tolerance at dt=tau/1000: the allowed interval error is 2 dt (1 + ISI_analytic/tau). This is a finite-step convergence check, not exact equality of continuous and discrete firing rates. Each of the eight conventions and configuration constants received a written verdict.

The supplied final campaign ledger contains 446 valid mutants, 437 kills, nine survivors and no timeouts. The operator grouping in Table 5 and file grouping in Table 9 reconcile when the C row is treated as an included subset. The rounded score is 98.0%. The earlier 464-mutant campaign used a different program state, so the score change is descriptive rather than a controlled comparison over identical mutants. Of the 437 recorded kills, 238 are assertion-failure outcomes and 199 are crash outcomes.

File	Mutants	Killed	Survived	Timed out	Score
qmodule/statevector_sim.py	231	227	4	0	98.3%
qmodule/qasm_module.py	21	21	0	0	100%
snn/lif_neuron.py	40	40	0	0	100%
gnn/message_passing.py	37	37	0	0	100%
fusion/shared_space.py	48	47	1	0	97.9%
orchestrator/pipeline.py	39	36	3	0	92.3%
orchestrator/wasm_bridge.py	18	17	1	0	94.4%
wasm-core/orchestrator_core.c	12	12	0	0	100%
Total	446	437	9	0	98.0%

Table 9. The final campaign against the revision-3 suite (v1.2). Zero timeouts: the binary search is gone.

The historical triage assigns accepted or config verdicts to all nine survivors. Those labels are judgments, not proofs of equivalence. The original integer seed formula is injective in tick and node index when 0<=index<1,000,003, but this does not cover unrestricted graph scale or prove independent streams. P10 observes whether sampled outputs change while embeddings also change; it does not inspect seeds or

directly establish their distinctness. The division mutant creates floating-point seeds and needs separate domain analysis. The deleted final-CDF guard remains a deterministically testable boundary case; the claimed failure probability near10^-16 is not established.

Surviving mutant	Original verdict	**Reason**
fusion: seed + idx → seed − idx	accepted	the specification is that distinct nodes get distinct sampling seeds; any injective derivation satisfies it
pipeline: seed + tick·1,000,003 → −; · → ÷; 1,000,003 → 1,000,004 (three mutants)	accepted	Historical rationale: distinct ticks should receive distinct seeds. P10 does not directly establish this; see the source review above.
wasm_bridge: subprocess timeout 30 → 31 s	config	a configuration constant
statevector_sim: parameter-expression length limit 200 → 201; > → ≥	config	a defensive limit and its boundary
statevector_sim: eval(safe, {"__builtins__": {}}, {}) with its two dictionaries swapped	accepted	the character whitelist is the sandbox; the empty builtins table is defense in depth, unobservable through the whitelist
statevector_sim: cumulative[−1] = 1.0 deleted	accepted	Reported defense against CDF rounding; per-shot failure probability is not established. Requires a controlled draw at the final-CDF boundary.

Table 10. The nine survivors of the final campaign and their verdicts, as recorded in method/triage.json and reproduced in BUILD_INDEX.md.

8.6 What the Mechanism Found, and What It Costs

The manuscript reports that the generated campaign exposed additional test gaps in gate implementation paths, nondefault LIF parameters, outputs and validation boundaries. These findings illustrate the value of a repeatable mutation scope, while their exact classification requires the raw campaign and baseline test sources. The operator set and generators were human choices; the mechanism broadens those choices' application rather than eliminating them.

The campaign covers a chosen operator set and eligible sites, with no higher-order mutation claim. Its score depends on test selection, crash and timeout classification, and survivor decisions. Mutating production source also does not independently validate test arithmetic. Runtime varies by machine and process overhead; the supplied historical ledger records its own elapsed time. A fresh run should preserve its source identity and all outcomes rather than overwriting the historical evidence.

9. Discussion

9.1 What "Honest Scaffolding" Means in Practice

The practical value of this work is that its claims can be traced to specific formulas, tests and experiment records. That traceability also makes errors visible: a correct local calculation can coexist with incorrect wiring, and a passing suite can coexist with an overbroad document claim. Revision 4 uses the same standard for the manuscript itself by correcting the table interpretation and distinguishing finite evidence from universal guarantees.

9.2 Where the QASM-as-WASM Analogy Holds and Where It Breaks

The analogy is useful as an interface boundary: an artifact is loaded, parameters are supplied, and a result contract is inspected. The classical LIF implementation and the statevector simulator have different roles, but the exact outputs currently consumed by the pipeline are deterministic classical calculations. A real-hardware extension would need to define how measurement uncertainty enters the embedding update and how its results are compared with the closed-form baseline.

Wasm's isolation boundary does not protect one allocation inside linear memory from another, nor does it by itself bound all resource consumption. The reported capacity guard addresses a specific demonstrated C-core overrun, and parser checks address specified rejection cases. These are useful fixes, but they do not establish complete memory safety or safe execution of arbitrary untrusted input. Such claims require the full input and resource contract, source review and targeted tests.

9.3 On the 4D Coordinate: Resisting the Temptation to Oversell Geometry

The output joins a three-dimensional random projection with elapsed simulation age. Its established mathematical identity is unbiased squared norm over projection randomness; the paper does not establish accurate distances for the fixed projection, graph-neighbor preservation or good partition quality. A sharding key based on these coordinates requires measurements of distortion, cut edges, balance and communication on representative graphs, together with a policy for never-spiked nodes.

9.4 On Testing the Tests

The composition example shows why a test should derive expected triggers and state transitions independently rather than accept the pipeline's own log as its oracle. Mutation testing helps probe that boundary. Automated mutation applies selected rules broadly and repeatably, but people still choose the rules, generators and acceptance criteria. Its useful question is specific: which supported incorrect behaviors would this suite detect, and which remain unresolved?

10. Toward Scale: A Staged Roadmap

10.1 M1–M5 Overview

The roadmap separates the supplied toy implementation from proposed larger deployments. M0 denotes that implementation and its tested scenarios, subject to the limitations in Section 11. No milestone beyond M0 is demonstrated here. Later milestones need their own workload, baseline, environment, acceptance criteria and evidence.

Milestone	Goal	Status
M0	Toy-scale reference implementation, every layer validated, composition validated per tick	Implemented at toy scope; see limitations
M1	Single-machine scale-up: sparse-matrix aggregation, millions of nodes	Proposed
M2	Persistent WASM runtime replacing per-tick subprocess spawning	Proposed
M3	Distributed graph sharding across machines	Proposed
M4	Real or GPU-accelerated quantum backend behind qmodule, with batched calls	Proposed
M5	Full smoke test at target scale, same validation methodology re-run	Proposed

Table 11. Proposed roadmap. M0 is implemented at the stated toy scope; its tests do not establish all-input correctness or production readiness. M1 through M5 remain proposals.

M1 should compare the existing gather/scatter operations with suitable sparse-matrix implementations. PyTorch Geometric documents both approaches [4]; a gated mean also needs the active-degree denominator, so a static adjacency multiply alone is insufficient. M2 is a persistent Wasm runtime with measured serialization, instantiation and end-to-end tick costs. M3 evaluates distributed infrastructure such as DistDGL [14] against representative graph structure rather than assuming good partitioning from the projected coordinates.

10.2 The Hardest Open Problem: Real Quantum Hardware at Billions-of-Triggers Volume

Consider a conditional serial-dispatch calculation: N nodes, activity fraction rho=10^-4, one job per trigger, and assumed end-to-end job latency L=1 ms. At N=10^10 this yields 10^6 triggers and N*rho*L=1,000 seconds per tick, about 16.7 minutes. At N=1.4x10^8 it yields 14,000 triggers and 14 seconds per tick. The arithmetic is correct under these assumptions. These figures are not measured QPU execution times or universal lower bounds; overhead, shots, concurrency and batching are unspecified. Real-time feasibility also requires a stated tick deadline. The present exact two-qubit outputs have a direct classical closed-form baseline, which should be evaluated before hardware scaling is proposed.

Direct cosine evaluation is the first baseline. Hardware or batched-simulation studies need a richer workload, defined observables, shots and error metrics. Batching can reduce dispatch overhead; qubit capacity, readout, total work and the tick deadline still need measurement.

11. Limitations

The implementation is a toy-scale architecture study with fixed or random weights, no training and no quantum-hardware experiment. Its weighted GraphSAGE layer is tested separately but is absent from the composed tick. The parser implements a restricted gate subset and does not model general measurement semantics. The LIF host starts a Node subprocess per call, and no production latency, energy or scale advantage is established. Zero-tolerance comparisons do not establish universal bitwise portability. The projection does not guarantee useful geometry for sharding. The reference statevector algorithm uses the production parser or parameter evaluator in parts of its frontend, so its independence is limited to the separately implemented evolution path. Source inspection also finds that the parameter evaluator's literal and bound-parameter fast paths return before the general finite-value check; its blanket finite-number contract therefore needs correction and targeted tests. Seed domains, accepted mutation survivors, CDF boundaries and safe handling of arbitrary untrusted input remain specific engineering questions. Magica simulator components and additional research fragments are outside this release unless separately supplied and cleared.

12. Conclusion

The supplied package implements a small hybrid pipeline and a method for examining its tests. Revision 4 repeats the current ten-module suite successfully, reconciles the mutation ledger, and sharpens the mathematical and architectural claims. The result is a reproducible starting point for further work with explicit limits, not a claim of quantum advantage, exhaustive verification or readiness at the proposed large scale.

The strongest contribution is the connection between an architectural claim, a separately motivated oracle, and a mutation that tests whether the oracle detects the relevant error. That connection is useful beyond this particular circuit. It remains conditional on the input domain, implementation version, frontend dependencies and survivor policy. New work should extend those records as carefully as it extends the code.

The next experiments should measure what the current evidence leaves open: clean rebuild reproducibility, frontend contract boundaries, persistent-runtime overhead, and richer circuit behavior against a matched classical baseline. Each result should retain its source identity, command, environment, raw outcome and limitations so that others can challenge and improve it.

References

[1] Scarselli, F., Gori, M., Tsoi, A. C., Hagenbuchner, M., & Monfardini, G. (2009). The Graph Neural Network Model. IEEE Transactions on Neural Networks, 20(1), 61-80. https://dl.acm.org/doi/abs/10.1109/tnn.2008.2005605

[2] Kipf, T. N., & Welling, M. (2017). Semi-Supervised Classification with Graph Convolutional Networks. International Conference on Learning Representations (ICLR 2017). https://arxiv.org/abs/1609.02907

[3] Hamilton, W. L., Ying, R., & Leskovec, J. (2017). Inductive Representation Learning on Large Graphs. Advances in Neural Information Processing Systems 30 (NeurIPS 2017). https://arxiv.org/abs/1706.02216

[4] Fey, M., & Lenssen, J. E. (2019). Fast Graph Representation Learning with PyTorch Geometric. ICLR 2019 Workshop on Representation Learning on Graphs and Manifolds. https://arxiv.org/abs/1903.02428

[5] Maass, W. (1997). Networks of spiking neurons: the third generation of neural network models. Neural Networks, 10(9), 1659-1671. https://www.sciencedirect.com/science/article/abs/pii/S0893608097000117

[6] Gerstner, W., & Kistler, W. M. (2002). Spiking Neuron Models: Single Neurons, Populations, Plasticity. Cambridge University Press. ISBN 0-521-89079-9.

[7] Davies, M., Srinivasa, N., Lin, T.-H., et al. (2018). Loihi: A Neuromorphic Manycore Processor with On-Chip Learning. IEEE Micro, 38(1), 82-99. https://ieeexplore.ieee.org/document/8259423

[8] Verdon, G., McCourt, T., Luzhnica, E., Singh, V., Leichenauer, S., & Hidary, J. (2019). Quantum Graph Neural Networks. arXiv:1909.12264. https://arxiv.org/abs/1909.12264

[9] Cerezo, M., Arrasmith, A., Babbush, R., et al. (2021). Variational Quantum Algorithms. Nature Reviews Physics, 3, 625-644. https://www.nature.com/articles/s42254-021-00348-9

[10] Cross, A. W., Bishop, L. S., Smolin, J. A., & Gambetta, J. M. (2017). Open Quantum Assembly Language. arXiv:1707.03429. https://arxiv.org/abs/1707.03429

[11] Quantinuum. (2024). Systems software updates: QEC Decoder Toolkit, December 2024; accompanying repetition-code and repeat-until-success examples. https://docs.quantinuum.com/product-updates/posts/systems_12_2024/ . Accessed 8 November 2026. Historical toolkit landing page: https://docs.quantinuum.com/systems/trainings/h2/getting_started/qec_decoder_toolkit/qec_decoder_toolkit.html .

[12] Haas, A., Rossberg, A., Schuff, D. L., Titzer, B. L., Holman, M., Gohman, D., Wagner, L., Zakai, A., & Bastien, J. F. (2017). Bringing the Web up to Speed with WebAssembly. Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2017). https://dl.acm.org/doi/10.1145/3062341.3062363

[13] Johnson, W. B., & Lindenstrauss, J. (1984). Extensions of Lipschitz mappings into a Hilbert space. Contemporary Mathematics, 26, 189-206. https://web.stanford.edu/class/cs114/readings/JL-Johnson.pdf

[14] Zheng, D., Ma, C., Wang, M., Zhou, J., Su, Q., Song, X., Gan, Q., Zhang, Z., & Karypis, G. (2020). DistDGL: Distributed Graph Neural Network Training for Billion-Scale Graphs. arXiv:2010.05337. https://arxiv.org/abs/2010.05337

[15] Berends, F. A., & Giele, W. T. (1988). Recursive calculations for processes with n gluons. Nuclear Physics B, 306(4), 759-808. https://www.sciencedirect.com/science/article/abs/pii/0550321388904427

[16] DeMillo, R. A., Lipton, R. J., & Sayward, F. G. (1978). Hints on Test Data Selection: Help for the Practicing Programmer. IEEE Computer, 11(4), 34-41. https://dl.acm.org/doi/10.1109/C-M.1978.218136

[17] Jia, Y., & Harman, M. (2011). An Analysis and Survey of the Development of Mutation Testing. IEEE Transactions on Software Engineering, 37(5), 649-678. https://dl.acm.org/doi/10.1109/TSE.2010.62

Revision 4 reproducibility and further work

Reproducibility record

On 8 November 2026, all ten supplied test modules completed with exit status zero, 1,286 passing check entries and zero failures under Python 3.12.14, NumPy 2.3.5 and Node 24.22.1 on Windows. The original 47 supplied files remained byte-identical during that suite run. The supplied WASM binary was executed, rather than rebuilt. These results reproduce the final suite on this environment; they do not reproduce the earlier baseline archives or every historical review probe.

Independent mathematical checks

A separate dense 4x4 calculation compared the two displayed Z expectations at 1,000 seeded angle triples in [-10,10]^3. The maximum absolute difference was 4.44e-16. Two hundred constant-current, monotone Euler-LIF cases agreed with the stated first-crossing formula; the maximum trajectory difference was 5.33e-15. These are new checks of the formulas, not substitute implementations of the entire scaffold.

Why node relabeling gives equivariance

Let H be node features, A the directed adjacency and P a node-permutation matrix. For a mean aggregator with correspondingly permuted degrees, M(PAP^T,PH)=P M(A,H). Shared feature weights and a shared rowwise activation commute with node permutation, so F(PAP^T,PH)=P F(A,H). This establishes the algebraic equivariance of that specified layer. It does not establish invariance of the nodewise output or correctness of every implementation; the tests check sampled instances.

Why the two-qubit primitive needs a classical baseline

After RY(a) and RY(b) act on |00>, the control Z expectation is cos(a). Conjugating the target Z observable through CX yields Z0 Z1, whose expectation on the product input is cos(a)cos(b). The final RZ commutes with those observables. Direct cosine evaluation therefore computes the consumed map without statevector storage or a QPU. A claimed quantum benefit would require a different workload and a measured comparison.

Priority experiments

First reproduce the build from C source with pinned compiler flags and compare the resulting binary and numerical behavior. Then add explicit tests for non-finite parameter fast paths, measurement placement, seed-domain collisions and the final-CDF boundary. Measure a persistent Wasm host against the existing subprocess host and direct NumPy under matched workloads, separating startup, serialization, kernel time and total tick latency. A quantum extension should specify new observables, shots and error metrics and compare them with the direct classical baseline. Report adverse and null results alongside improvements.

Authorship and AI assistance

Aaron Michael Weis is the author identified for this release, ORCID 0009-0002-1906-148X. Earlier manuscripts attributed drafting and development review to Claude models. Revision 4 used Codex and two Astra Ultra review passes for mathematical, source and reference review, followed by local execution checks. This assistance is disclosed separately from human authorship and does not imply independent human peer review. Exact historical model-version labels have not been independently authenticated.

Availability and reuse

Repository: https://github.com/OneiroNexus/QASM-WASM . Code is offered under the MIT License. This manuscript is licensed under Creative Commons Attribution 4.0 International (CC BY 4.0), with attribution to Aaron Michael Weis and a link to https://creativecommons.org/licenses/by/4.0/ . Existing third-party credits remain with their respective works. No Zenodo DOI is asserted until a deposit has been completed.